



AVIGNON
UNIVERSITÉ

Rapport – Chapitre 6

Algorithmes Quantiques : Deutsch-Jozsa, Grover et Dürr-Høyer

Patrice SEBASTIANO

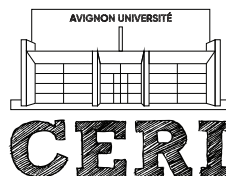
16/06/2026

**Master d'Informatique
Informatique Quantique**

UE Stage de Recherche – LIA Avignon

Encadrant
Serigne GUEYE

UFR
SCIENCES
TECHNOLOGIES
SANTÉ



CENTRE
D'ENSEIGNEMENT
ET DE RECHERCHE
EN INFORMATIQUE
ceri.univ-avignon.fr

Sommaire

6	Algorithmes Quantiques	3
6.1	Algorithme de Deutsch-Jozsa	3
6.1.1	Problème	3
6.1.2	Circuit	3
6.1.3	Implémentation	3
6.1.4	Résultats	4
6.2	Algorithme de Grover	5
6.2.1	Problème	5
6.2.2	Structure des registres	5
6.2.3	Vue d'ensemble du circuit	5
6.2.4	Encodage des données	5
6.2.5	L'oracle NOT-XOR	6
6.2.6	Le diffuseur de Grover	8
6.2.7	Résultats de simulation	9
6.2.8	Test sur matériel réel IBM	9
6.3	Analyse Mathématique de Grover et Preuve de Convergence (BBHT)	9
1.	Initialisation et Opérateurs dans la vision de Grover	9
2.	Réduction dimensionnelle – cas $t = 1$ (Approche Boudin)	11
2bis.	Généralisation au cas t solutions et réécriture trigonométrique	11
3.	Expression du diffuseur 2D et invariance de la récurrence	12
5.	Discussion sur la convergence de BBHT	12
5bis.	Preuve de convergence par l' <i>Exponential Random Scaling</i> quand $1 \leq t \leq \frac{3}{4}N$	13
6.4	Recherche du Minimum Quantique – Algorithme de Dürr-Høyer	15
6.4.1	Problème et formulation	15
6.4.2	Circuit quantique	15
6.4.3	Implémentation de l'ERS	16
6.4.4	Résultats de simulation	16
6.4.5	Non-convergence stochastique – cas seed=88	17
A	Calcul de la somme $\sum_{j=0}^{m-1} \cos((4j+2)\theta)$	18
B	Nombre espéré d'itérations au stade s	18

6 Algorithmes Quantiques

Ce rapport présente l'implémentation de deux algorithmes quantiques suivants : Deutsch-Jozsa et Grover, implémentés en Qiskit 2.x, accompagnée de leurs circuits dessinés en $\text{\LaTeX}$ avec la bibliothèque `quantikz`.

6.1 Algorithme de Deutsch-Jozsa

6.1.1 Problème

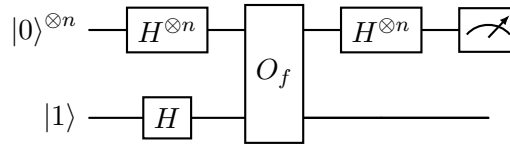
Est donné une fonction booléenne $f : \{0,1\}^n \rightarrow \{0,1\}$, et les définitions suivantes :

$$f(x) = \begin{cases} c \in \{0,1\} & \forall x \in \{0,1\}^n \quad \textbf{(constante)} \\ 0 \text{ ou } 1 \text{ avec } |\{x \mid f(x) = 0\}| = |\{x \mid f(x) = 1\}| = 2^{n-1} & \textbf{(équilibrée)} \\ \text{ni constante ni équilibrée} & \textbf{(autre)} \end{cases}$$

Le but est de déterminer à quelle catégorie appartient f .

Un algorithme classique déterministe nécessite $2^{n-1} + 1$ requêtes dans le pire cas. L'algorithme de Deutsch-Jozsa répond en **une seule requête quantique**.

6.1.2 Circuit



Le registre ancilla est préparé dans $|-\rangle$ pour activer le *phase kickback*. Après l'oracle et la transformée de Hadamard inverse, la règle de décision est :

$$\begin{aligned} \text{mesure} = \mathbf{0}^n &\iff f \text{ est constante.} \\ \text{mesure} \neq \mathbf{0}^n, \text{ toutes mesures présentes} &\iff f \text{ est équilibrée.} \\ \text{sinon, } f &\text{ n'est ni constante ni équilibrée.} \end{aligned}$$

6.1.3 Implémentation

L'implémentation teste toutes les fonctions binaires jusqu'à $n = N_{\max} = 3$ bits. La valeur $N_{\max} = 4$ a été écartée car elle génère $4^4 = 256$ oracles, ce qui demande un temps d'exécution trop long.

La partie critique est la génération procédurale des oracles. `itertools.product([0,1], repeat=2n)` énumère les 2^{2^n} tables de vérité possibles. Pour chaque table, on construit le circuit correspondant.

La difficulté vient de la porte MCX : elle se déclenche uniquement quand *tous* ses qubits de contrôle valent $|1\rangle$. Or, x est la représentation binaire de l'indice de la ligne courante dans la table de vérité, et certains de ses bits valent 0. Pour chaque ligne où $f(x) = 1$, on applique une porte X sur chacun des qubits d'entrée dont le bit correspondant dans x vaut 0 (ce qui les amène à $|1\rangle$, permettant le déclenchement de la MCX), puis on remet ces mêmes qubits à $|0\rangle$ après la MCX pour ne pas corrompre les autres termes de la superposition :

```
1 def generate_oracles(n):
2     for k, truth_table in enumerate(itertools.product([0,1], repeat=2**n)):
3         qc = QuantumCircuit(n + 1, name=f"Oracle_f_{k}")
4         target = n
5         for x, fx in enumerate(truth_table):
6             if fx == 1:
7                 bits = format(x, f'0{n}b')
8                 # flip qubits where bit=0 so MCX fires on this input
9                 for i, b in enumerate(reversed(bits)):
10                     if b == '0':
11                         qc.x(i)
12                 qc.mcx(list(range(n)), target)
13                 # uncompute
14                 for i, b in enumerate(reversed(bits)):
15                     if b == '0':
16                         qc.x(i)
```

6.1.4 Résultats

Le circuit a été exécuté sur les 2^{2^n} oracles pour $n \in \{1, 2, 3\}$. Les résultats obtenus sont les suivants :

	$n = 1$	$n = 2$	$n = 3$
Constantes	2	2	2
Équilibrées	2	6	70
Autres	0	8	184
Total	4	16	256

Le nombre de fonctions constantes est toujours 2 ($f \equiv 0$ et $f \equiv 1$), quel que soit n . Le nombre de fonctions équilibrées vaut $\binom{2^n}{2^{n-1}}$ (choisir quelles 2^{n-1} entrées reçoivent la valeur 1), ce qui donne bien 2, 6 et 70 pour $n = 1, 2, 3$. Toutes les autres fonctions sont classifiées dans la catégorie *autre* – elles ne sont ni constantes ni équilibrées, et l'algorithme de Deutsch-Jozsa les détecte correctement comme telles.

6.2 Algorithme de Grover

6.2.1 Problème

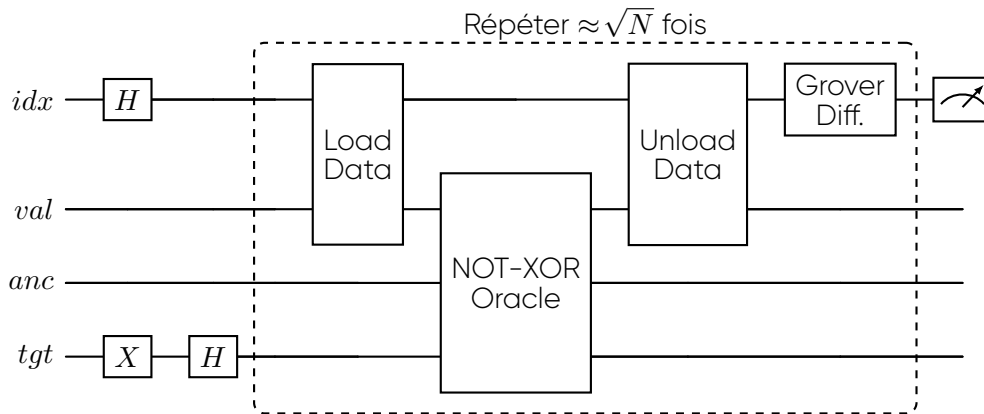
On dispose d'une liste non ordonnée de $N = 2^n$ entiers stockés dans un DataFrame Pandas. On cherche l'indice i^* d'une valeur cible v^* . L'algorithme de Grover résout ce problème en $O(\sqrt{N})$ requêtes, alors qu'en informatique classique, il faut $O(N)$ dans le pire des cas.

6.2.2 Structure des registres

Le circuit utilise quatre registres :

Registre	Taille	Rôle
idx	n qubits	Superposition des indices – mesuré en sortie
val	n_{val} qubits	Représentation binaire de v_i – intriqué avec idx lors du chargement
anc	n_{val} qubits	Ancilla pour la comparaison bit à bit
tgt_phase	1 qubit	Préparé dans $ -\rangle$ pour le phase kickback

6.2.3 Vue d'ensemble du circuit



Le nombre d'itérations optimal est :

$$k = \left\lfloor \frac{\pi}{4} \sqrt{\frac{N}{k_{\text{cibles}}}} \right\rfloor.$$

6.2.4 Encodage des données

L'étape de chargement associe à chaque indice en superposition la valeur correspondante dans la liste. Après cette étape, l'état du système est :

$$\frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |i\rangle |v_i\rangle,$$

ce qui constitue une *représentation quantique du DataFrame* : chaque indice i est intriqué avec la valeur v_i qui lui est associée.

Construction procédurale. Le circuit de chargement est généré dynamiquement à partir du DataFrame : pour chaque ligne i , on applique une porte MCX par bit de v_i égal à 1, contrôlée sur l'état $|i\rangle$ du registre **idx**.

Par exemple, pour la liste suivante (indice sur 2 bits, valeur sur 3 bits) :

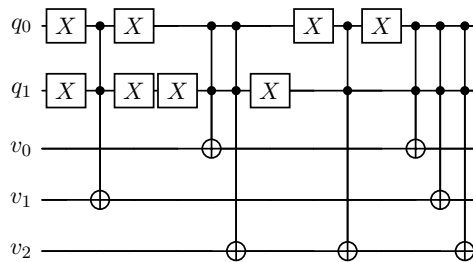
Indice		Valeur	
déc	bin	déc	bin
0	00	2	010
1	01	5	101
2	10	4	100
3	11	7	111

Données d'entrée

$$\frac{1}{2}(|00\rangle|010\rangle + |01\rangle|101\rangle + |10\rangle|100\rangle + |11\rangle|111\rangle)$$

État quantique après chargement

Les portes X avant et après chaque MCX retournent temporairement les qubits de contrôle à $|1\rangle$ pour les bits de l'indice qui valent 0 :



Circuit de chargement des données pour les 4 indices

Chaque indice $|i\rangle$ est désormais intriqué avec la valeur $|v_i\rangle$ correspondante dans la liste.

Déchargement des données. Après l'oracle, le registre `val` doit être remis à $|0\rangle$, faute de quoi le diffuseur de Grover ne produirait pas de résultats corrects. Comme toutes les portes quantiques sont unitaires, il suffit de réappliquer les mêmes portes MCX dans l'ordre inverse – ce qui annule l'encodage et restaure $|i\rangle|v_i\rangle \rightarrow |i\rangle|0\rangle$.

Convention `ctrl_state` en Qiskit

Dans Qiskit, la porte MCX avec état de contrôle explicite s'utilise ainsi :

```
1 qc.mcx(
2     control_qubits=index_qubits, # liste de qubits, qubit 0 = LSB
3     target_qubit=target_val_qubit,
4     ctrl_state=idx_bin_qiskit, # chaîne MSB en premier, ex. "110" pour 6
5 )
```

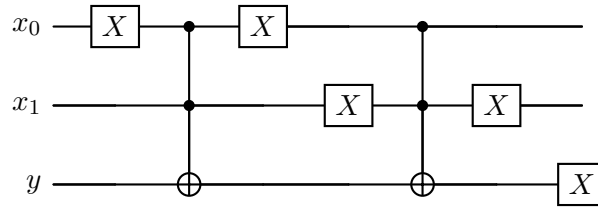
Attention aux conventions opposées des deux arguments : `control_qubits` suit la convention little-endian de Qiskit (qubit 0 = LSB), tandis que `ctrl_state` est une chaîne binaire standard (MSB en premier), convertie en interne par `int(string, 2)`. Les mêmes bits sont donc écrits dans des ordres opposés au sein du même appel : pour l'indice 6 ("110"), on a `ctrl_state="110"` mais `control_qubits=[q0, q1, q2]` où `q0` porte le bit de poids faible.

6.2.5 L'oracle NOT-XOR

Pour trouver un élément dans la liste, l'oracle doit comparer bit à bit la valeur chargée dans `val` à la valeur cible v^* . On a donc besoin d'un circuit qui implémente cette comparaison entre deux chaînes de bits.

Dans le cas élémentaire – comparer deux bits x_0 et x_1 – on veut obtenir 1 si et seulement si $x_0 = x_1$, et 0 sinon. C'est exactement la porte **NOT-XOR** (ou XNOR) :

$$\text{NOT-XOR}(x_0, x_1) = 1 \iff x_0 = x_1.$$



Rappel – qubit ancilla. On remarque ici un troisième qubit y alors que l'on ne comparait que deux bits x_0 et x_1 . Il s'agit d'un qubit *ancilla* : il ne fait pas partie des données d'entrée, mais sert de registre auxiliaire pour mémoriser le résultat. En sortie du circuit, $y = 1$ si $x_0 = x_1$, et $y = 0$ sinon.

Généralisation à des chaînes de n bits. Pour comparer deux chaînes de n bits $x^{(1)} = x_0^{(1)} x_1^{(1)} \dots x_{n-1}^{(1)}$ et $x^{(2)} = x_0^{(2)} x_1^{(2)} \dots x_{n-1}^{(2)}$, on applique la porte NOT-XOR indépendamment à chaque position i , avec un qubit ancilla y_i dédié :

$$y_i = \text{NOT-XOR}(x_i^{(1)}, x_i^{(2)}) = 1 \iff x_i^{(1)} = x_i^{(2)}.$$

Les deux chaînes sont égales si et seulement si tous les y_i valent 1 simultanément.

Application à l'algorithme de Grover.

Choix de n_{val} . La taille des chaînes n'est pas fixée a priori : elle est choisie dynamiquement comme le nombre minimal de bits pour représenter en binaire le plus grand entier de la liste :

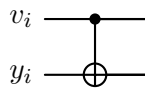
$$n_{\text{val}} = \lceil \log_2(\max_i(v_i) + 1) \rceil.$$

Simplification de NOT-XOR. Rappelons le problème : on cherche une valeur v^* connue à l'avance dans une liste non ordonnée. Parce qu'elle est connue avant le démarrage de l'algorithme, v^* peut être encodée explicitement en binaire à la construction du circuit. On n'a donc pas à comparer deux chaînes en superposition quantique : on compare le registre **val** (quantique, contenant toutes les valeurs du DataFrame en superposition) à v^* (classique, une chaîne de bits fixée).

Puisque v^* est une valeur binaire classique, cette comparaison se réduit à deux cas distincts pour chaque bit i . De plus, le bit cible v_i^* étant une constante, il n'est plus nécessaire d'utiliser un troisième qubit pour le représenter : le circuit passe de 3 qubits (x_0, x_1, y) à seulement 2 qubits (v_i, y_i) .

Cas $v_i^* = 1$

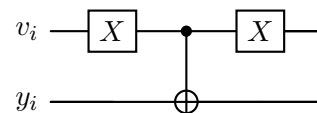
On veut $y_i = 1$ ssi $v_i = 1$: un simple CNOT suffit.



Le CNOT se déclenche quand $v_i = 1$, y_i vaut donc 1 exactement quand $v_i = v_i^*$.

Cas $v_i^* = 0$

On veut $y_i = 1$ ssi $v_i = 0$: on entoure le CNOT de deux portes X .



Le premier X retourne v_i ; le CNOT se déclenche quand $v_i = 0$; le second X restaure v_i .

Lorsque tous les y_i valent 1 (correspondance complète), une porte MCX applique le phase kickback sur **tgt_phase**, puis les ancillas sont décomputés pour restaurer **anc** dans $|0\rangle$.

L'implémentation concrète ci-dessous reçoit la représentation binaire de v^* et construit le circuit correspondant à la volée :

```

1 def oracle(qc, target_bin_value):
2     qregs_by_name = {reg.name: reg for reg in qc.qregs}
3     qr_value = qregs_by_name["val"]
4     qr_ancilla = qregs_by_name["anc"]
5     qr_target = qregs_by_name["tgt_phase"]
6
7     target_bin_qiskit = target_bin_value[::-1] # little-endian: qubit 0 = LSB
8
9     # Comparaison bit a bit : adapte la porte selon le bit cible
10    for bit_position, bit in enumerate(target_bin_qiskit):
11        if bit == "1":
12            qc.cx(qr_value[bit_position], qr_ancilla[bit_position])
13        else: # bit == "0"
14            qc.x(qr_value[bit_position])
15            qc.cx(qr_value[bit_position], qr_ancilla[bit_position])
16            qc.x(qr_value[bit_position])
17
18    # Phase kickback : tgt_phase recoit le -1 si toutes les ancillas = 1
19    qc.mcx(list(qr_ancilla), qr_target[0])
20
21    # Uncompute : memes portes en ordre inverse pour remettre anc a |0>
22    for bit_position, bit in reversed(list(enumerate(target_bin_qiskit))):
23        if bit == "1":
24            qc.cx(qr_value[bit_position], qr_ancilla[bit_position])
25        else:
26            qc.x(qr_value[bit_position])
27            qc.cx(qr_value[bit_position], qr_ancilla[bit_position])
28            qc.x(qr_value[bit_position])

```

La fonction s'adapte automatiquement à toute valeur cible : la chaîne binaire de v^* est renversée (convention little-endian de Qiskit, qubit 0 = LSB), puis chaque bit est traité indépendamment par le bloc `if/else`. Changer la cible ne modifie pas la structure du code – seule la suite de portes générée change.

6.2.6 Le diffuseur de Grover

Le diffuseur implémente $2|s\rangle\langle s| - I$ où $|s\rangle = H^{\otimes n}|0\rangle$:

$$H^{\otimes n} \rightarrow X^{\otimes n} \rightarrow \underbrace{H_t \rightarrow \text{MCX} \rightarrow H_t}_{\text{MCZ}} \rightarrow X^{\otimes n} \rightarrow H^{\otimes n}.$$

L'empilement $H \rightarrow \text{MCX} \rightarrow H$ autour du qubit cible t convertit la porte MCX (flip de bit) en MCZ (flip de phase), ce qui permet d'amplifier correctement l'amplitude.

Condition préalable : désentremêler les registres. Pour que le diffuseur fonctionne correctement, il doit recevoir un état où *seul* le registre `idx` porte l'information utile – à savoir la marque de phase -1 sur les indices cibles. Tout entanglement avec `val` ou `anc` perturbera l'amplification d'amplitude, ce qui produira des probabilités incorrectes.

L'unique rôle de la séquence *Load Data* $\rightarrow$ *Oracle* $\rightarrow$ *Unload Data* est donc d'affecter le signe -1 sur les indices cibles, sans laisser aucune trace dans les autres registres. Les deux étapes d'uncompute servent exactement à cela :

- **Dans l'oracle** : les ancillas `anc` sont décomputés après le phase kickback, remettant chaque `anc[i]` à $|0\rangle$ et supprimant l'entanglement entre `anc` et `val/idx`.
- **Après l'oracle** : le registre `val` est déchargé (*Unload Data*) avant l'application du diffuseur, restaurant $|v_i\rangle \rightarrow |0\rangle$ et supprimant l'entanglement entre `val` et `idx`.

À l'entrée du diffuseur : `val` et `anc` sont revenus à $|0\rangle$ (plus d'entanglement avec `idx`), `tgt_phase` reste dans son état initial $|-\rangle$ (inchangé, il n'a servi qu'à induire le phase kickback), et `idx` seul porte les marques de phase : les amplitudes des indices cibles ont été multipliées par -1 . Le diffuseur peut donc opérer uniquement sur `idx` comme prévu.

6.2.7 Résultats de simulation

Pour $n = 3$ bits ($N = 8$ entrées, valeurs aléatoires dans $[0,50)$) :

Valeur cible	Indices attendus	Indice trouvé	Correct
14	[2]	2	✓
20	[5]	5	✓
38	[0, 6]	0	✓
38	[0, 6]	0	✓
42	[3]	3	✓

La probabilité de succès théorique pour $N = 8$, une cible, deux itérations est :

$$P = \sin^2\left(5 \cdot \arcsin \frac{1}{\sqrt{8}}\right) \approx 97.5 \%$$

6.2.8 Test sur matériel réel IBM

Le circuit a été soumis sur un backend IBM Quantum (156 qubits, `ibm_kingston`) via `qiskit-ibm-runtime` et `SamplerV2`. Le résultat est une distribution quasi-uniforme sur les huit indices.

Explication. Le circuit possède 16 qubits et des dizaines de portes MCX. Après transpilation en portes CX natives, la profondeur atteint plusieurs centaines de portes. Avec un taux d'erreur de $\sim 0.1 \%$ par porte, le circuit se décohère entièrement avant de terminer.

Cette limitation est fondamentale à l'ère NISQ : la recherche de Grover sur une base de données arbitraire nécessite de la QRAM (Quantum RAM), dont l'implémentation pratique n'existe pas à ce jour. Le résultat erroné n'est donc pas un bug d'implémentation mais une contrainte physique inhérente aux dispositifs actuels.

6.3 Analyse Mathématique de Grover et Preuve de Convergence (BBHT)

Cette section présente les fondements mathématiques de l'algorithme de Grover – en partant de l'article original de 1996, en passant par l'approche géométrique 2D exposée dans Boudin – pour aboutir à la preuve de convergence de l'extension *Exponential Random Scaling* établie par Boyer, Brassard, Høyer et Tapp (BBHT). L'objectif est de montrer que l'algorithme reste efficace même lorsque le nombre de solutions t est inconnu.

1. Initialisation et Opérateurs dans la vision de Grover Dans son article original de 1996, Lov Grover définit l'état initial par une superposition uniforme de tous les états de la base computationnelle via l'application de la transformée de Hadamard $H^{\otimes n} |0\rangle$. L'amplitude initiale de chaque état est identique :

$$|\psi_H\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} |k\rangle \implies k_0 = \ell_0 = \frac{1}{\sqrt{N}}$$

où k_0 désigne l'amplitude de l'unique solution cible, et ℓ_0 l'amplitude de chacune des $N - 1$ non-solutions.

Remarque. L'indice k_0 est au singulier car l'article original de Grover traite le cas d'une *unique* solution dans la base de données – c'est précisément l'hypothèse fondatrice de son travail

de 1996. Par ailleurs, l'égalité $k_0 = \ell_0 = \frac{1}{\sqrt{N}}$ traduit le fait qu'à l'initialisation le système est *parfaitement symétrique* : toutes les amplitudes sont rigoureusement identiques. La distinction entre k_0 (amplitude de la solution) et ℓ_0 (amplitude d'une non-solution) est donc purement nominale à ce stade ; c'est l'oracle qui brisera cette symétrie à la première itération en inversant le signe de k_0 .

Grover définit ensuite le diffuseur comme l'opérateur d'inversion autour de la moyenne $D = 2|\psi_H\rangle\langle\psi_H| - I$. Cet opérateur se traduit par une matrice carrée d'ordre N dont les coefficients valent :

$$D_{ij} = \frac{2}{N} \quad \text{pour } i \neq j \quad \text{et} \quad D_{ii} = -1 + \frac{2}{N}$$

En appliquant D à un état dont l'unique solution cible a l'amplitude k_j et chacun des $N - 1$ états restants a l'amplitude ℓ_j , Grover (Theorem 2, 1996) démontre que les amplitudes à l'itération suivante vérifient la récurrence :

$$k_{j+1} = \left(\frac{2}{N} - 1\right) k_j + 2 \frac{N-1}{N} \ell_j$$

$$\ell_{j+1} = \frac{2}{N} k_j + \frac{N-2}{N} \ell_j$$

Ces deux équations sont la traduction directe de l'action de D sur les amplitudes : le premier terme de chaque ligne correspond au coefficient diagonal $D_{ii} = \frac{2}{N} - 1$, et le second au coefficient hors-diagonal $D_{ij} = \frac{2}{N}$ multiplié par le nombre d'états de même type.

Conclusion de Grover (Theorem 3, 1996). Grover établit ensuite que si l'amplitude de la solution satisfait $0 < k_j < \frac{1}{\sqrt{2}}$ et que $\ell_j > 0$, alors l'incrément d'amplitude lors d'une itération vérifie :

$$\Delta k > \frac{1}{2\sqrt{N}}$$

et ℓ reste positif après l'itération. Il en découle directement qu'après au plus $M < \sqrt{2N}$ itérations, k dépasse le seuil $\frac{1}{\sqrt{2}}$. Or la probabilité de mesurer la solution est k^2 : dès que $k > \frac{1}{\sqrt{2}}$, on a $k^2 > \frac{1}{2}$. Une mesure effectuée à ce moment donne la solution avec une probabilité supérieure à $\frac{1}{2}$.

C'est là que s'arrête l'article de Grover : en $O(\sqrt{N})$ itérations, la probabilité de succès franchit $\frac{1}{2}$. C'est un résultat fondateur, mais qui repose sur deux hypothèses implicites : (i) il existe exactement une solution ($t = 1$), et (ii) on s'arrête au bon moment, c'est-à-dire avant que la probabilité ne redescende par *over-cooking*.

Extension par Boyer, Brassard, Høyer et Tapp (BBHT, 1998). L'article BBHT lève ces deux contraintes en apportant deux contributions :

- une formule explicite pour le nombre optimal d'itérations j^* lorsque t est connu ;
- une méthode de tirage pseudo-aléatoire du nombre d'itérations – l'*Exponential Random Scaling* (ERS) – qui garantit la convergence en $O(\sqrt{N/t})$ même lorsque t est inconnu.

Ces deux résultats seront détaillés en fin de section. Avant cela, il est nécessaire d'opérer une réécriture trigonométrique de l'état de Grover, qui servira de socle commun aux deux démonstrations.

Présentation dans ce rapport : apport de Boudin. Dans leur article original, BBHT utilisent déjà implicitement une description géométrique de l'état de Grover dans un plan 2D, avec un angle θ tel que $\sin(\theta) = \sqrt{t/N}$. Ce cadre trigonométrique est antérieur à l'ouvrage de Boudin. Cependant, c'est à travers l'exposition pédagogique de Boudin que j'ai construit ma compréhension de ces résultats. La reformulation présentée dans les paragraphes suivants est donc celle de Boudin.

2. Réduction dimensionnelle – cas $t = 1$ (Approche Boudin) La réduction repose sur une observation clé : toutes les non-solutions jouent un rôle strictement identique dans l'algorithme. On peut donc les regrouper en un unique vecteur normalisé $|\chi\rangle$, et ne retenir que deux axes – la solution $|k_0\rangle$ et ce vecteur collectif – pour décrire l'évolution complète de l'état :

$$|\chi\rangle = \frac{1}{\sqrt{N-1}} \sum_{k \neq k_0} |k\rangle$$

Par linéarité, l'état de superposition uniforme initial $|\psi_H\rangle$ peut être réécrit dans cette nouvelle base orthonormée $(|\chi\rangle, |k_0\rangle)$:

$$|\psi_H\rangle = \left(\sqrt{\frac{N-1}{N}} \right) |\chi\rangle + \left(\frac{1}{\sqrt{N}} \right) |k_0\rangle$$

À ce stade précis, avant toute considération trigonométrique, les coordonnées globales dans ce plan 2D sont $c_0 = \sqrt{\frac{N-1}{N}}$ (coordonnée sur l'axe des non-solutions) et $k_0 = \frac{1}{\sqrt{N}}$ (coordonnée sur l'axe de la solution).

L'amplitude individuelle de Grover pour chaque non-solution ℓ_0 reste accessible de manière sous-jacente par la relation de normalisation : $c_0 = \ell_0 \sqrt{N-1} = \frac{1}{\sqrt{N}} \sqrt{N-1}$.

2bis. Généralisation au cas t solutions et réécriture trigonométrique Lorsque la base de données contient t solutions parmi N états, on définit deux vecteurs normalisés analogues à $|k_0\rangle$ et $|\chi\rangle$:

$$|\beta\rangle = \frac{1}{\sqrt{t}} \sum_{\substack{x \\ f(x)=1}} |x\rangle, \quad |\alpha\rangle = \frac{1}{\sqrt{N-t}} \sum_{\substack{x \\ f(x)=0}} |x\rangle$$

L'état initial se réécrit dans la base $(|\alpha\rangle, |\beta\rangle)$:

$$|\psi_H\rangle = \sqrt{\frac{N-t}{N}} |\alpha\rangle + \sqrt{\frac{t}{N}} |\beta\rangle$$

On introduit alors l'angle θ défini par :

$$\sin \theta = \sqrt{\frac{t}{N}}, \quad \cos \theta = \sqrt{\frac{N-t}{N}}$$

ce qui permet d'écrire de façon compacte :

$$|\psi_H\rangle = \cos \theta |\alpha\rangle + \sin \theta |\beta\rangle$$

Le cas $t = 1$ du paragraphe précédent est un cas particulier : $\sin \theta = \frac{1}{\sqrt{N}}$, $|\alpha\rangle = |\chi\rangle$ et $|\beta\rangle = |k_0\rangle$.

On adopte à présent les notations de BBHT : k_j désigne l'amplitude de *chaque* état solution individuel, et ℓ_j celle de chaque état non-solution. En substituant les définitions de $|\alpha\rangle$ et $|\beta\rangle$, l'état initial se réécrit directement dans la base computationnelle :

$$|\psi_H\rangle = \cos \theta |\alpha\rangle + \sin \theta |\beta\rangle = \frac{\cos \theta}{\sqrt{N-t}} \sum_{\substack{x \\ f(x)=0}} |x\rangle + \frac{\sin \theta}{\sqrt{t}} \sum_{\substack{x \\ f(x)=1}} |x\rangle = \sum_{\substack{x \\ f(x)=0}} \ell_0 |x\rangle + \sum_{\substack{x \\ f(x)=1}} k_0 |x\rangle$$

ce qui identifie directement :

$$k_0 = \frac{1}{\sqrt{t}} \sin \theta \quad \text{et} \quad \ell_0 = \frac{1}{\sqrt{N-t}} \cos \theta$$

Pour $t = 1$ et $\sin \theta = 1/\sqrt{N}$, on retrouve $k_0 = \ell_0 = 1/\sqrt{N}$, ce qui coïncide exactement avec les amplitudes initiales de Grover (paragraphe 1) : les deux notations sont donc cohérentes. Ce sont ces définitions qu'emploie BBHT dans sa preuve.

3. Expression du diffuseur 2D et invariance de la récurrence En appliquant l'opérateur $D = 2|\psi_H\rangle\langle\psi_H| - I$ directement sur les vecteurs de notre base 2D, et en injectant la définition géométrique $|\psi_H\rangle = \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix}$, le projecteur se calcul explicitement par produit matriciel :

$$2|\psi_H\rangle\langle\psi_H| - I = 2 \begin{pmatrix} \cos^2(\theta) & \cos(\theta)\sin(\theta) \\ \sin(\theta)\cos(\theta) & \sin^2(\theta) \end{pmatrix} - \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

En utilisant les identités de l'angle double ($\cos(2\theta) = 2\cos^2(\theta) - 1 = 1 - 2\sin^2(\theta)$ et $\sin(2\theta) = 2\sin(\theta)\cos(\theta)$), le diffuseur se réduit à :

$$D_{2D} = \begin{pmatrix} \cos(2\theta) & \sin(2\theta) \\ \sin(2\theta) & -\cos(2\theta) \end{pmatrix}$$

Une itération complète de Grover combinant l'Oracle $O = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ et ce diffuseur donne l'opérateur $G = D_{2D} \times O$:

$$G = \begin{pmatrix} \cos(2\theta) & -\sin(2\theta) \\ \sin(2\theta) & \cos(2\theta) \end{pmatrix}$$

Il s'agit de la matrice de rotation standard dans le plan. Par conséquent, l'angle total croît de 2θ à chaque itération, et les amplitudes individuelles au pas j sont données par induction :

$$k_j = \frac{1}{\sqrt{t}} \sin((2j+1)\theta) \quad \text{et} \quad \ell_j = \frac{1}{\sqrt{N-t}} \cos((2j+1)\theta)$$

La probabilité de mesurer une solution après j itérations est donc $t k_j^2 = \sin^2((2j+1)\theta)$.

Remarque (conflit de notation θ vs $\theta/2$). Il existe une divergence de convention dans la littérature : dans Boudin, l'angle initial satisfait $\sin(\theta_{\text{Boudin}}/2) = 1/\sqrt{N}$, de sorte qu'une itération tourne de θ_{Boudin} ; dans BBHT, $\sin \theta = 1/\sqrt{N}$ directement, et une itération tourne de 2θ . Les deux sont équivalentes ($\theta_{\text{Boudin}} = 2\theta_{\text{BBHT}}$). Ce rapport suit la convention BBHT.

5. Discussion sur la convergence de BBHT On cherche à maximiser k_j , car $t k_j^2 = \sin^2((2j+1)\theta)$ est la probabilité de mesurer une solution après j itérations. Le maximum de $\sin^2$ est atteint quand son argument vaut $\frac{\pi}{2}$, c'est-à-dire quand :

$$(2j+1)\theta = \frac{\pi}{2} \implies j^* = \frac{\pi - 2\theta}{4\theta} = \left\lfloor \frac{\pi}{4\theta} \right\rfloor$$

Avec $\sin \theta = \sqrt{t/N}$, on a pour $t \ll N$ l'approximation $\theta \approx \sqrt{t/N}$, d'où :

$$j^* \approx \frac{\pi}{4} \sqrt{\frac{N}{t}}$$

Lorsque t est connu – et en particulier dans le cas original de Grover où $t = 1$ – cette formule donne directement le nombre optimal d'itérations : $j^* \approx \frac{\pi}{4} \sqrt{N}$, ce qui reproduit la complexité en $O(\sqrt{N})$.

Avant de passer à la preuve pour t inconnu, précisons les cas limites traités séparément par BBHT :

- $t = 1$ (**Grover classique**). La formule $j^* = \lfloor \pi/(4\theta) \rfloor$ avec $\sin \theta = 1/\sqrt{N}$ fournit le nombre optimal d'itérations de façon explicite, ce qui est le résultat établi dans les paragraphes précédents.
- $t = 0$ (**aucune solution**). BBHT montre qu'on peut détecter ce cas avec une précision arbitraire ; je renvoie le lecteur à l'article original, ce cas n'ayant pas été creusé dans ce rapport.
- $t > \frac{3}{4}N$ (**majorité de solutions**). Dès qu'une fraction supérieure à $3/4$ des états est une solution, un simple tirage aléatoire classique trouve une solution avec probabilité $> 3/4$ en une requête. Le quantique n'apporte aucun avantage ; ce cas est donc exclu.

On se place donc dans le régime $1 \leq t \leq \frac{3}{4}N$ pour lequel l'approche quantique est pertinente.

5bis. Preuve de convergence par l'*Exponential Random Scaling* quand $1 \leq t \leq \frac{3}{4}N$

La stratégie de BBHT est une preuve de convergence *statistique* : plutôt que de chercher le nombre exact d'itérations optimales, on montre qu'un tirage pseudo-aléatoire du nombre d'itérations garantit une probabilité de succès minorée par une constante. Traiter chaque tentative comme un tirage indépendant permet ensuite de conclure sur la convergence globale.

Probabilité de succès après j itérations. Soit j le nombre d'itérations effectuées. Après j itérations, l'état du système quantique s'écrit dans la base $(|\alpha\rangle, |\beta\rangle)$:

$$|\psi_j\rangle = \sum_{f(x)=0} \ell_j |x\rangle + \sum_{f(x)=1} k_j |x\rangle$$

où k_j et ℓ_j sont les amplitudes de chaque état individuel. La probabilité de mesurer une solution est la somme des carrés des amplitudes sur les t états solution :

$$P(\text{succès}) = \sum_{f(x)=1} k_j^2 = t k_j^2 = t \cdot \frac{\sin^2((2j+1)\theta)}{t} = \sin^2((2j+1)\theta)$$

Probabilité totale par la loi des probabilités totales. Lorsque t est inconnu, on ne peut pas calculer j^* . L'idée de BBHT est de modéliser le choix du nombre d'itérations comme une expérience aléatoire : on introduit la variable aléatoire J tirée *uniformément* dans $\{0, 1, \dots, m-1\}$, de sorte que :

$$\forall j \in \{0, \dots, m-1\}, \quad P(J = j) = \frac{1}{m}$$

La probabilité de succès *conditionnellement* au tirage $J = j$ est celle calculée plus haut :

$$P(\text{succès} \mid J = j) = \sin^2((2j+1)\theta)$$

La probabilité *totale* de succès s'obtient alors par la loi des probabilités totales, en sommant sur toutes les issues possibles de J :

$$P_m = \sum_{j=0}^{m-1} P(J = j) \cdot P(\text{succès} \mid J = j) = \frac{1}{m} \sum_{j=0}^{m-1} \sin^2((2j+1)\theta)$$

On développe $\sin^2$ à l'aide de l'identité trigonométrique $\sin^2(x) = \frac{1 - \cos(2x)}{2}$:

$$P_m = \frac{1}{m} \sum_{j=0}^{m-1} \frac{1 - \cos(2(2j+1)\theta)}{2} = \frac{1}{2} - \frac{1}{2m} \sum_{j=0}^{m-1} \cos((4j+2)\theta)$$

La somme des cosinus est une somme géométrique que l'on évalue par décomposition en exponentielles complexes (voir annexe A) :

$$\sum_{j=0}^{m-1} \cos((4j+2)\theta) = \frac{\sin(4m\theta)}{2\sin(2\theta)}$$

On obtient ainsi la forme close :

$$P_m = \frac{1}{2} - \frac{\sin(4m\theta)}{4m\sin(2\theta)}$$

On se place dans le cas $m \geq \frac{1}{\sin(2\theta)}$. Comme $|\sin(4m\theta)| \leq 1$, on peut majorer le terme oscillatoire :

$$\frac{\sin(4m\theta)}{4m\sin(2\theta)} \leq \frac{1}{4m\sin(2\theta)} \leq \frac{1}{4}$$

où la dernière inégalité découle directement de $m \geq 1/\sin(2\theta)$, qui donne $4m\sin(2\theta) \geq 4$. On obtient donc :

$$P_m = \frac{1}{2} - \frac{\sin(4m\theta)}{4m\sin(2\theta)} \geq \frac{1}{2} - \frac{1}{4} = \frac{1}{4}$$

C'est cette minoration $P_m \geq \frac{1}{4}$ qui est le résultat clé. Ce seuil critique $m^* = 1/\sin(2\theta)$ joue un rôle central dans la preuve de convergence de l'algorithme qui suit.

Algorithme : Exponential Random Scaling (BBHT). On met momentanément m^* de côté et on pose l'algorithme. On choisit $\lambda \in \mathbb{R}$ avec $\lambda > 1$; BBHT prennent $\lambda = 6/5$ (toute valeur strictement entre 1 et $4/3$ convient). On initialise $m = 1$.

1. Initialiser $m = 1$ et fixer $\lambda = 6/5$.
2. Tirer j uniformément au hasard parmi les entiers $\{0, 1, \dots, [m] - 1\}$.
3. Appliquer j itérations de l'algorithme de Grover à partir de l'état initial $|\Psi_0\rangle = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |i\rangle$.
4. Mesurer le registre ; soit i le résultat obtenu.
5. Si $T[i] = x$, le problème est résolu : **fin**.
6. Sinon, poser $m \leftarrow \min(\lambda m, \sqrt{N})$ et retourner à l'étape 2.

La borne $m \leftarrow \min(\lambda m, \sqrt{N})$ assure que m croît de façon exponentielle (d'où le nom) sans jamais dépasser $\sqrt{N}$, qui est l'ordre de grandeur de $m_0 = 1/\sin(2\theta) \approx \sqrt{N/(4t)}$ dans le régime $t \ll N$.

Théorème (BBHT, 1998). L'algorithme ERS trouve une solution en temps espéré $O(\sqrt{N/t})$.

Preuve. Soit θ l'angle tel que $\sin^2 \theta = t/N$. On pose le seuil critique :

$$m_0 = \frac{1}{\sin(2\theta)} = \frac{N}{2\sqrt{(N-t)t}} < \sqrt{\frac{N}{t}}$$

où l'inégalité finale vient de l'hypothèse $t \leq 3N/4$.

Coût espéré pour atteindre le stade critique. Au s -ième passage dans la boucle principale, la valeur courante de m est λ^{s-1} . Comme j est tiré uniformément dans $\{0, \dots, [m] - 1\}$, le nombre espéré d'itérations de Grover est inférieur à $\lambda^{s-1}/2$ (voir annexe B). On dit que l'algorithme *atteint le stade critique* s'il effectue plus de $\lceil \log_\lambda m_0 \rceil$ tours de boucle ; à ce moment, m a dépassé m_0 . En effet, à l'entrée du stade s , on a $m = \lambda^{s-1}$, donc $m \geq m_0$ dès que $s - 1 \geq \log_\lambda m_0$, soit à partir du stade $s = \lceil \log_\lambda m_0 \rceil + 1$. La base λ est naturelle ici car

m est multiplié par λ à chaque tour : la suite (m) est une suite géométrique de raison λ , et $\lceil \log_\lambda m_0 \rceil$ est précisément le nombre de tours nécessaires pour que m franchisse le seuil m_0 . Le nombre total espéré d'itérations pour atteindre le stade critique est au plus :

$$\frac{1}{2} \sum_{s=1}^{\lceil \log_\lambda m_0 \rceil} \lambda^{s-1} < \frac{1}{2} \cdot \frac{\lambda}{\lambda-1} m_0 = 3m_0$$

où l'on a utilisé $\lambda/(\lambda-1) = 6$ pour $\lambda = 6/5$. Ainsi, si l'algorithme réussit *avant* d'atteindre le stade critique, il le fait en temps $O(m_0) = O(\sqrt{N/t})$.

Coût espéré après le stade critique. Une fois le stade critique atteint, $m \geq m_0 = 1/\sin(2\theta)$, donc par la minoration établie précédemment, chaque tour de boucle réussit avec probabilité au moins $1/4$. Notons u le nombre de tours supplémentaires après le stade critique avant de trouver une solution; on a $P(u \geq k) \leq (3/4)^k$. En notant que la valeur de m au $(u + \lceil \log_\lambda m_0 \rceil)$ -ième tour vaut $\lambda^{u+\lceil \log_\lambda m_0 \rceil-1} \leq \lambda^u m_0$, le coût espéré après le stade critique est majoré par :

$$\frac{1}{2} \sum_{u=0}^{\infty} \frac{3^u}{4^{u+1}} \lambda^u m_0 = \frac{m_0}{8} \sum_{u=0}^{\infty} \left(\frac{3\lambda}{4}\right)^u = \frac{m_0}{8} \cdot \frac{1}{1-3\lambda/4} = \frac{\lambda}{8-6\lambda} m_0 = \frac{3}{2} m_0$$

où la série converge car $3\lambda/4 = 9/10 < 1$ pour $\lambda = 6/5$.

Conclusion. En sommant les deux contributions, le nombre total espéré d'itérations de Grover est au plus $3m_0 + \frac{3}{2}m_0 = \frac{9}{2}m_0$, ce qui est en $O(m_0) = O(\sqrt{N/t})$.

Pour $t \ll N$, on a $\frac{9}{2}m_0 \approx \frac{9}{4}\sqrt{N/t}$, soit moins de quatre fois le nombre d'itérations qu'aurait nécessité l'algorithme de Grover si t avait été connu à l'avance. $\square$

6.4 Recherche du Minimum Quantique – Algorithme de Dürr-Høyer

6.4.1 Problème et formulation

L'algorithme de Grover, dans sa forme originale, cherche un élément *connu* dans une base non ordonnée. Dürr et Høyer (1996) l'ont étendu à la *recherche du minimum* d'une fonction de coût $g : \{0,1\}^n \rightarrow \mathbb{R}$ sans connaître la valeur du minimum à l'avance. La preuve de convergence de l'ERS établie à la section précédente garantit que cet algorithme trouve le minimum en temps $O(\sqrt{N/t})$ espéré, où t désigne le nombre courant de solutions (états x tels que $g(x) < g(y)$).

On s'intéresse ici à des fonctions quadratiques binaires de la forme

$$g(x) = x^\top G x, \quad x \in \{0,1\}^n,$$

où G est une matrice de coefficients triangulaire supérieure. La première variable $x_0 = 1$ est toujours fixée (terme constant), ce qui laisse $n-1$ variables libres et un espace de recherche de taille $N = 2^{n-1}$.

6.4.2 Circuit quantique

Le circuit est organisé en quatre registres :

Registre	Taille	Rôle
x	$n-1$ qubits	Candidat courant (superposition)
y	$n-1$ qubits	Meilleur point connu (encodé classiquement)
scalar	m qubits	Calcul de $g(x) - g(y)$ en base de phase (QFT)
anc	1 qubit	Phase kickback via $ -\rangle$

L'oracle $O_{g,y}$ réalise les étapes suivantes pour chaque itération de Grover :

1. Encodage classique de y dans le registre y .
2. Mur de Hadamard sur **scalar** $\rightarrow$ base de phase.
3. Application des rotations de phase encodant $g(x) - g(y)$ (via des portes $P(\theta)$ et MCP(θ) contrôlées par x et y).
4. QFT $^{-1}$ sur **scalar** $\rightarrow$ base computationnelle ; le registre contient maintenant l'entier $g(x) - g(y)$ en complément à deux.
5. Phase kickback : CX(MSB de **scalar**, **anc**). Le MSB est à $|1\rangle$ si et seulement si $g(x) - g(y) < 0$, c'est-à-dire $g(x) < g(y)$.
6. QFT + inversion des phases + mur de Hadamard pour restaurer **scalar** à $|0\rangle$.

Le diffuseur est appliqué sur le registre x uniquement, car c'est lui qui est en superposition et dont on cherche à amplifier les amplitudes des solutions.

6.4.3 Implémentation de l'ERS

Le nombre de solutions t est inconnu et varie à chaque étape. On applique donc directement la technique ERS de la section 6.3 : à chaque tour, on tire j uniformément dans $\{0, 1, \dots, [m]\}$ et on applique j itérations de Grover.

- **Succès** ($g(x) < g(y)$) : on met à jour $y \leftarrow x$ et on remet $m \leftarrow 1$.
- **Échec** : $m \leftarrow \min(\lambda m, \sqrt{N})$ avec $\lambda = 6/5$.
- **Arrêt** : quand m atteint $\sqrt{N}$, on s'arrête avec le meilleur y trouvé.

6.4.4 Résultats de simulation

Cinq instances ont été testées avec des dimensions $n \in \{3, 4, 5, 6, 4\}$ et des coefficients aléatoires à diagonale dominante (pour garantir l'unicité du minimum). Le tableau ci-dessous résume les résultats.

Seed	n	$N = 2^{n-1}$	m qubits	Sauts GAS	$g(y^*)$ trouvé	Minimum global
42	3	4	6	1	-13.0	-13.0 ✓
101	4	8	6	2	-13.0	-13.0 ✓
7	5	16	8	3	33.0	33.0 ✓
88	6	32	9	0	46.0	35.0 ×
2026	4	8	6	2	-4.0	-4.0 ✓

Les figures 1 et 2 illustrent deux cas de convergence réussie. La trajectoire rouge (GAS) descend vers le minimum global en effectuant des sauts.

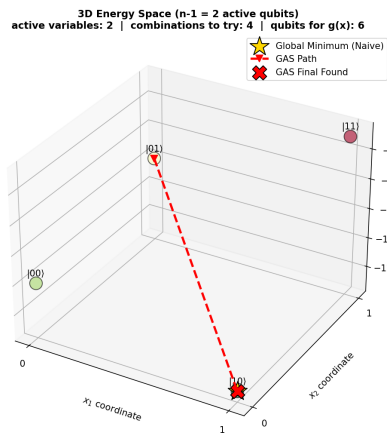


Figure 1. Cas $n = 3$, seed=42 : espace 3D, 4 combinaisons. GAS trouve le minimum global en 1 saut depuis le point de départ $|01\rangle$.

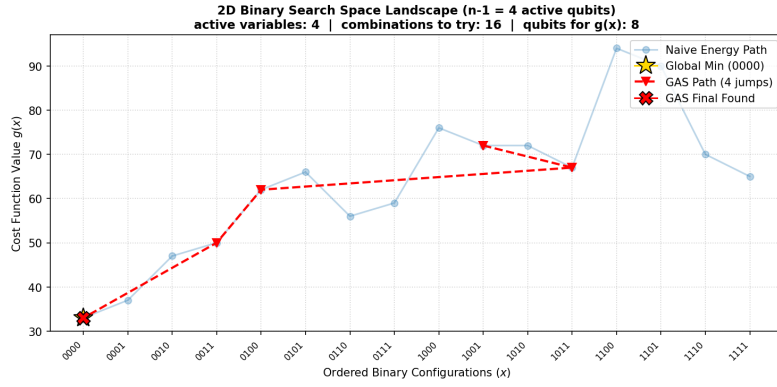


Figure 2. Cas $n = 5$, seed=7 : espace 2D, 16 combinaisons. GAS effectue 3 sauts successifs et converge vers le minimum global $|0000\rangle$ ($g = 33$).

6.4.5 Non-convergence stochastique – cas seed=88

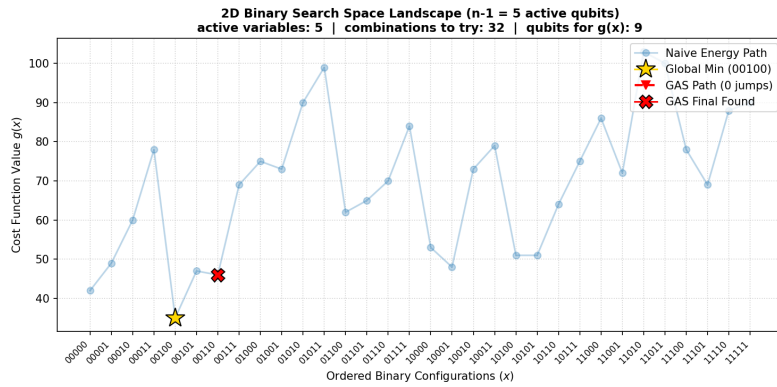


Figure 3. Cas $n = 6$, seed=88 : espace de 32 combinaisons. GAS n'effectue aucun saut et s'arrête avec $g(y) = 46$, loin du minimum global $g = 35$ ($|00100\rangle$).

Le cas seed=88 (figure 3) illustre un phénomène inhérent à l'algorithme : la **non-convergence stochastique**. L'algorithme tourne en boucle (m monte de 1.0 à $\sqrt{32} \approx 5.66$ par paliers de $\times 1.2$, soit ~ 10 tours) sans jamais mesurer un x tel que $g(x) < g(y_0) = 46$.

Ce n'est pas un bug : avec $t \approx 4$ solutions sur $N = 32$ (12 % de l'espace), la probabilité d'échec à chaque tour est $\geq 3/4$, et la probabilité de tout rater sur k tours consécutifs est $(3/4)^k$. Pour $k = 10$ tours, cela donne $(3/4)^{10} \approx 5.6\%$ – un événement rare mais possible. C'est précisément ce que garantit le théorème BBHT : la convergence est assurée en espérance, non presque sûrement.

La solution préconisée par Dürr et Høyer est de relancer l'algorithme avec un nouveau y_0 aléatoire dès que m atteint $\sqrt{N}$: chaque redémarrage est indépendant et la probabilité de rater tous les redémarrages décroît exponentiellement.

A Calcul de la somme $\sum_{j=0}^{m-1} \cos((4j+2)\theta)$

On souhaite calculer $S = \sum_{j=0}^{m-1} \cos((4j+2)\theta)$.

Passage aux exponentielles complexes. En utilisant $\cos(\varphi) = \operatorname{Re}(e^{i\varphi})$, on écrit :

$$S = \operatorname{Re} \left(\sum_{j=0}^{m-1} e^{i(4j+2)\theta} \right) = \operatorname{Re} \left(e^{2i\theta} \sum_{j=0}^{m-1} e^{4ij\theta} \right)$$

Somme géométrique. La somme intérieure est une suite géométrique de raison $r = e^{4i\theta}$ et de premier terme 1 :

$$\sum_{j=0}^{m-1} e^{4ij\theta} = \frac{1 - e^{4im\theta}}{1 - e^{4i\theta}}$$

On factorise numérateur et dénominateur par leurs demi-angles :

$$= \frac{e^{2im\theta} (e^{-2im\theta} - e^{2im\theta})}{e^{2i\theta} (e^{-2i\theta} - e^{2i\theta})} = \frac{e^{2im\theta} \cdot (-2i \sin(2m\theta))}{e^{2i\theta} \cdot (-2i \sin(2\theta))} = \frac{e^{2im\theta} \sin(2m\theta)}{e^{2i\theta} \sin(2\theta)}$$

Calcul de la partie réelle. En réinjectant dans S :

$$S = \operatorname{Re} \left(e^{2i\theta} \cdot \frac{e^{2im\theta} \sin(2m\theta)}{e^{2i\theta} \sin(2\theta)} \right) = \operatorname{Re} \left(\frac{e^{2im\theta} \sin(2m\theta)}{\sin(2\theta)} \right) = \frac{\sin(2m\theta) \cos(2m\theta)}{\sin(2\theta)}$$

En appliquant l'identité de l'angle double $\sin(2m\theta) \cos(2m\theta) = \frac{\sin(4m\theta)}{2}$, on obtient :

$$S = \sum_{j=0}^{m-1} \cos((4j+2)\theta) = \frac{\sin(4m\theta)}{2 \sin(2\theta)}$$

B Nombre espéré d'itérations au stade s

Au stade s de l'algorithme ERS, le paramètre vaut $m = \lambda^{s-1}$. L'indice j est tiré uniformément dans $\{0, 1, \dots, \lceil m \rceil - 1\}$, qui contient $\lceil m \rceil$ valeurs. Par la formule de Gauss ($\sum_{k=0}^{n-1} k = n(n-1)/2$), le nombre espéré d'itérations de Grover est :

$$\mathbb{E}[j] = \frac{0 + 1 + \dots + (\lceil m \rceil - 1)}{\lceil m \rceil} = \frac{\lceil m \rceil (\lceil m \rceil - 1)/2}{\lceil m \rceil} = \frac{\lceil m \rceil - 1}{2}.$$

Or $\lceil m \rceil - 1 < m$ dans tous les cas (strictement, que m soit entier ou non), donc :

$$\mathbb{E}[j] = \frac{\lceil m \rceil - 1}{2} < \frac{m}{2} = \frac{\lambda^{s-1}}{2}.$$